

¹University of Connecticut · ²San Francisco State University · ³UC Riverside · ⁴National University of Singapore · ⁵University of Pittsburgh · ⁶Florida State University · * Equal contribution

! Motivation

Standard random token masking:

- **CORE INSIGHT**

|| Main Results (Pass@1 %)

Relative Gain
HumanEval+ (T=512)

HumanEval Pass@1
(vs 39.6% baseline)

† From EvalPlus Leaderboard. HE = HumanEval, HE+ = HumanEval+.

Ablation: Masking Granularity

Key finding: Span-level masking corrupts entire subtrees — too coarse. TreeDiff’s **token-level** AST masking surgically targets high-influence nodes while preserving context.

 Contact Authors

Yiming Zeng

✉ yiming.zeng@uconn.edu

🔍 Generation Trajectory Analysis

```

groups = paren_string.split()
results = []
def parse_group(group):
    str_ => int :
        Parse a group of parentheses and return the deepest level of nesting.
    ***
    level = 0
    for char in group:
        if char == '(':
            level += 1
        elif char == ')':
            level -= 1
            if level == 0:
                return group
    for group in groups:
        results.append(parse_group(group))
max([len(r) for r in results])
if __name__ == '__main__':
    main()

```

Random Mask Baseline: Uses shallow counter heuristic, fails on nested structures. Fragmented generation.

TreeDiff: Establishes stack-based control flow (`stack.pop()`) early.
Correctly captures recursive nesting.

<> Generated Code Comparison

HumanEval/6 – Nested parentheses depth:

```
TreeDiff
for char in group:
    if char == '(':
        stack.append(char)
        max_depth = max(
            max_depth, len(stack)
        )
    elif char == ')':
        if stack: stack.pop()
```

```
Random Masking
count = 0
for char in group:
    if char == '(':
        count += 1
        # Fails to track
        # nested peak depth
```

★ Key Contributions

- **First AST-aware masking** for DLLMs in code generation
- **13.3% relative improvement** on HumanEval+ vs random masking
- Trained on **150K** long code reasoning samples
- Negligible overhead: AST parsing is $O(L)$

Conclusion

TreeDiff addresses a fundamental limitation: **random masking ignores code structure**. By aligning corruption with AST hierarchy, the model learns programming language compositionality, reconstructing programs that respect grammatical boundaries and capture long-range dependencies.

Impact: Structural priors unlock DLLMs for code — TreeDiff narrows the gap between diffusion and autoregressive paradigms.

Setup: LLaDA-8B backbone · 8× NVIDIA A100 · LoRA ($r=64, \alpha=128$) · 4096 max tokens · LR 5×10^{-6} ·
Benchmarks: HumanEval/+, MBPP/+

! Motivation

Standard random token masking:

- **CORE INSIGHT**

Method: AST-Guided Masking

TreeDiff parses code into an **Abstract Syntax Tree (AST)** and uses it to guide which tokens to mask — targeting structurally critical nodes.

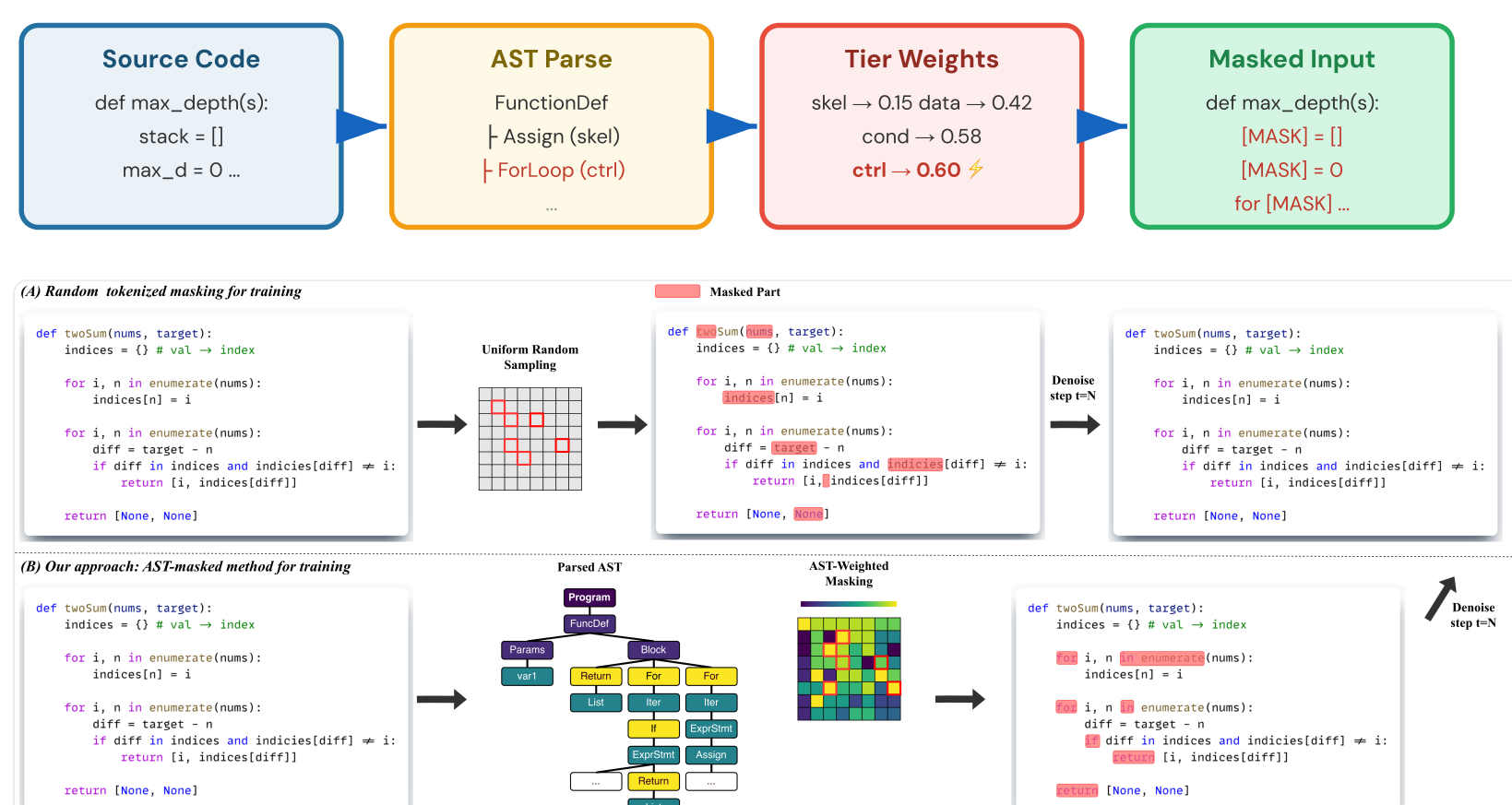
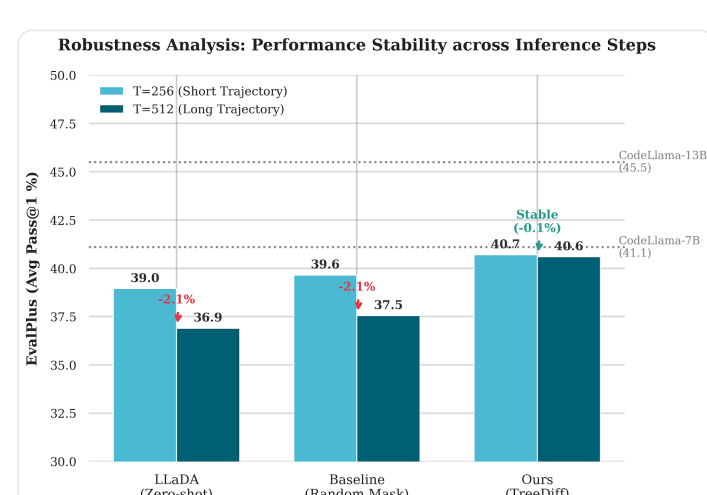


Figure 1: Uniform Random Masking (top) vs. AST-Weighted Sampling Masking (bottom).

AST Node Weighting

Category	Examples	Weight	Purpose
Skeletal	Imports, Constants	0.15	Preserve skeleton
Data Flow	Assigns, Calls	0.42	Moderate priority
Conditionals	if, elif, else	0.58	Core logic
Control Flow	for, while, return	0.60	⚡ Highest priority

Robustness Analysis



TreeDiff maintains stable performance (-0.1%) as T increases 256→512, while baselines degrade by ~2.1%.